



Training Multilayer Perceptron

CS-4025: Deep Learning for Perception

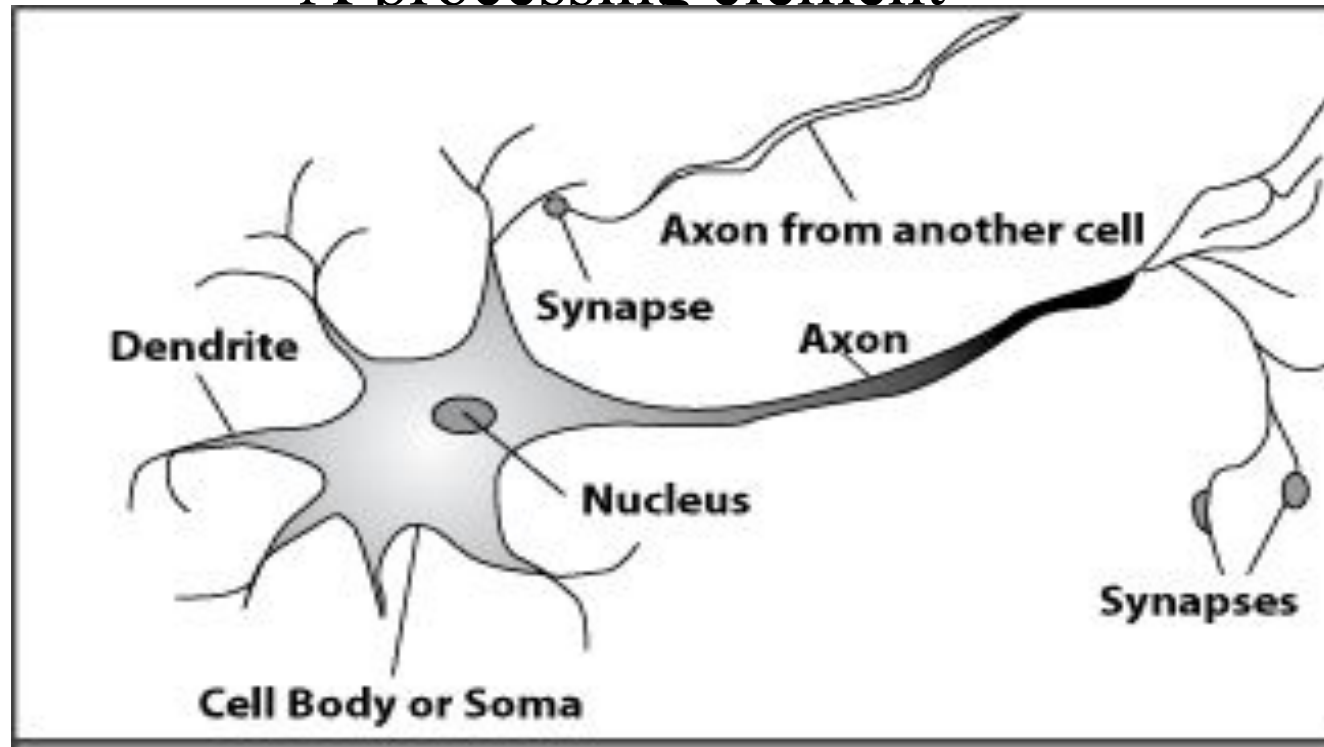
Goals

- Review of Previous Lecture
- Today's Lecture
 - Multilayer Perceptron
 - Backpropagation for training Neural Networks

Recap of Previous Lecture

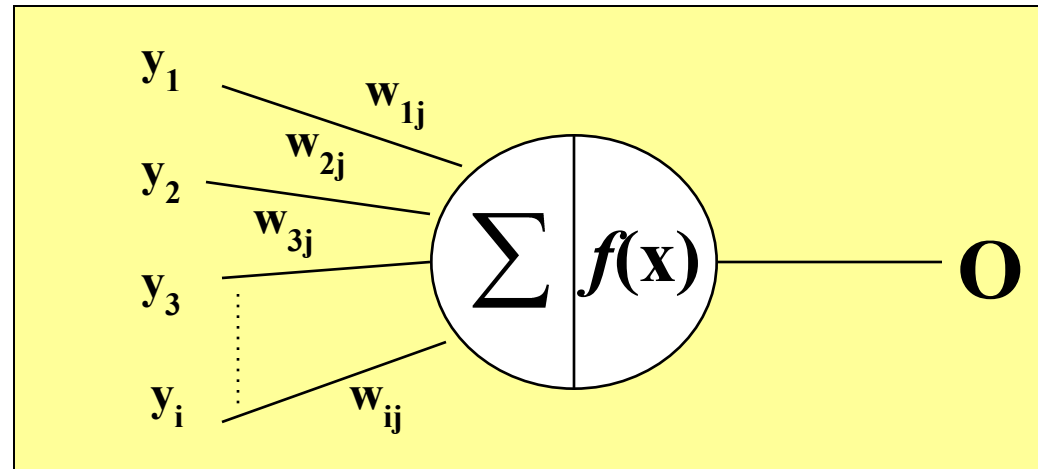
How do our brains work?

- A processing element



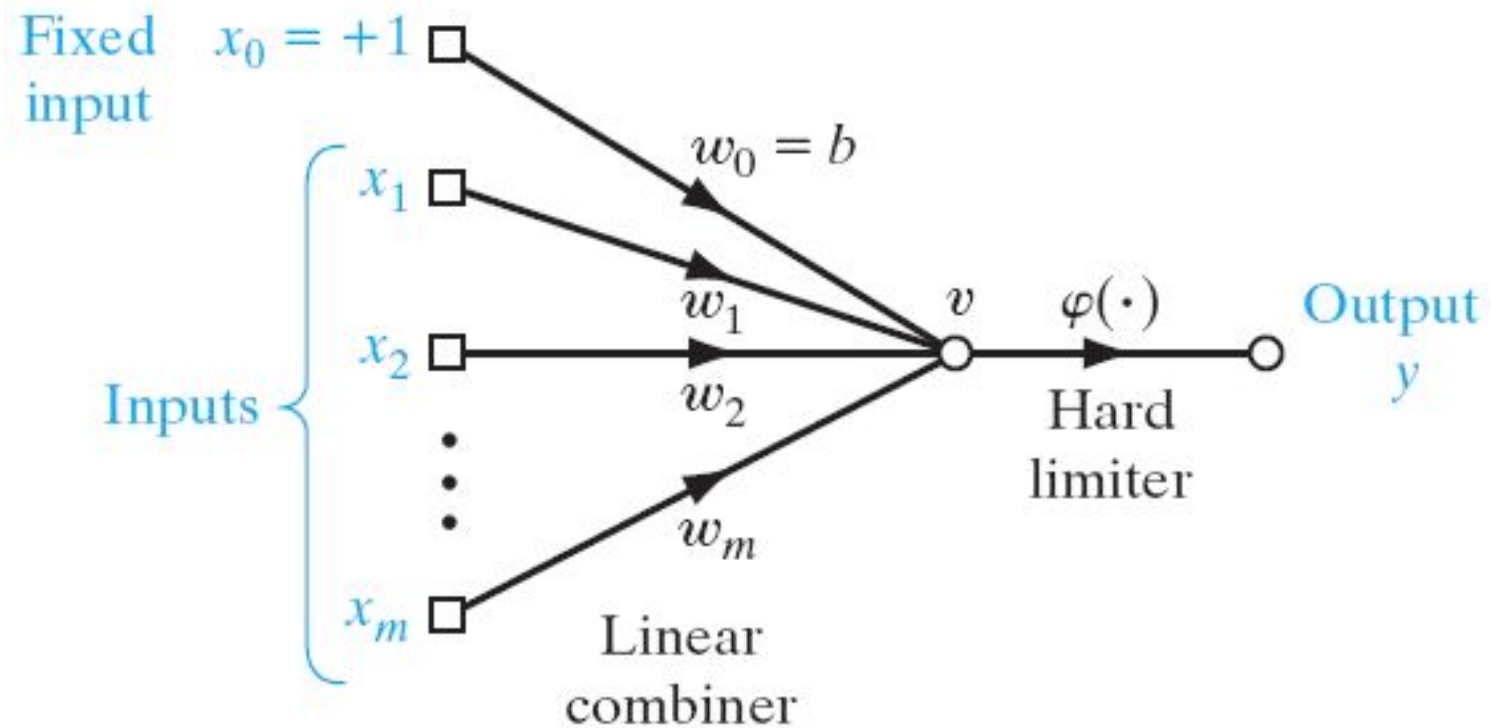
Dendrites: Input
Cell body: Processor
Synaptic: Link
Axon: Output

An Artificial Neuron

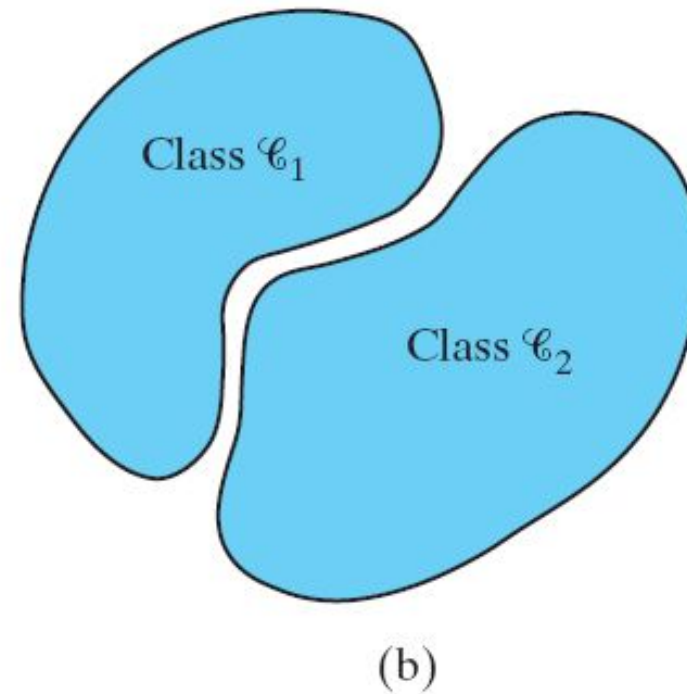
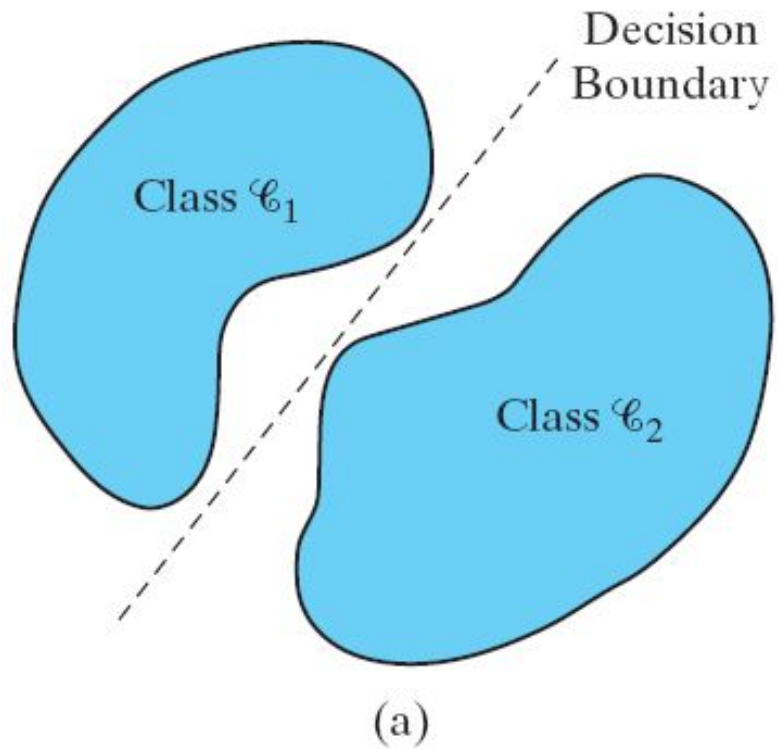


- Each neuron has weighted input connections from each of the units in the preceding layer.
- The unit performs a weighted sum of its inputs, to give its activation level.
- Activation level is passed through a sigmoid activation function to determine output.

Rosenblatt's perceptron model



Rosenblatt's perceptron model

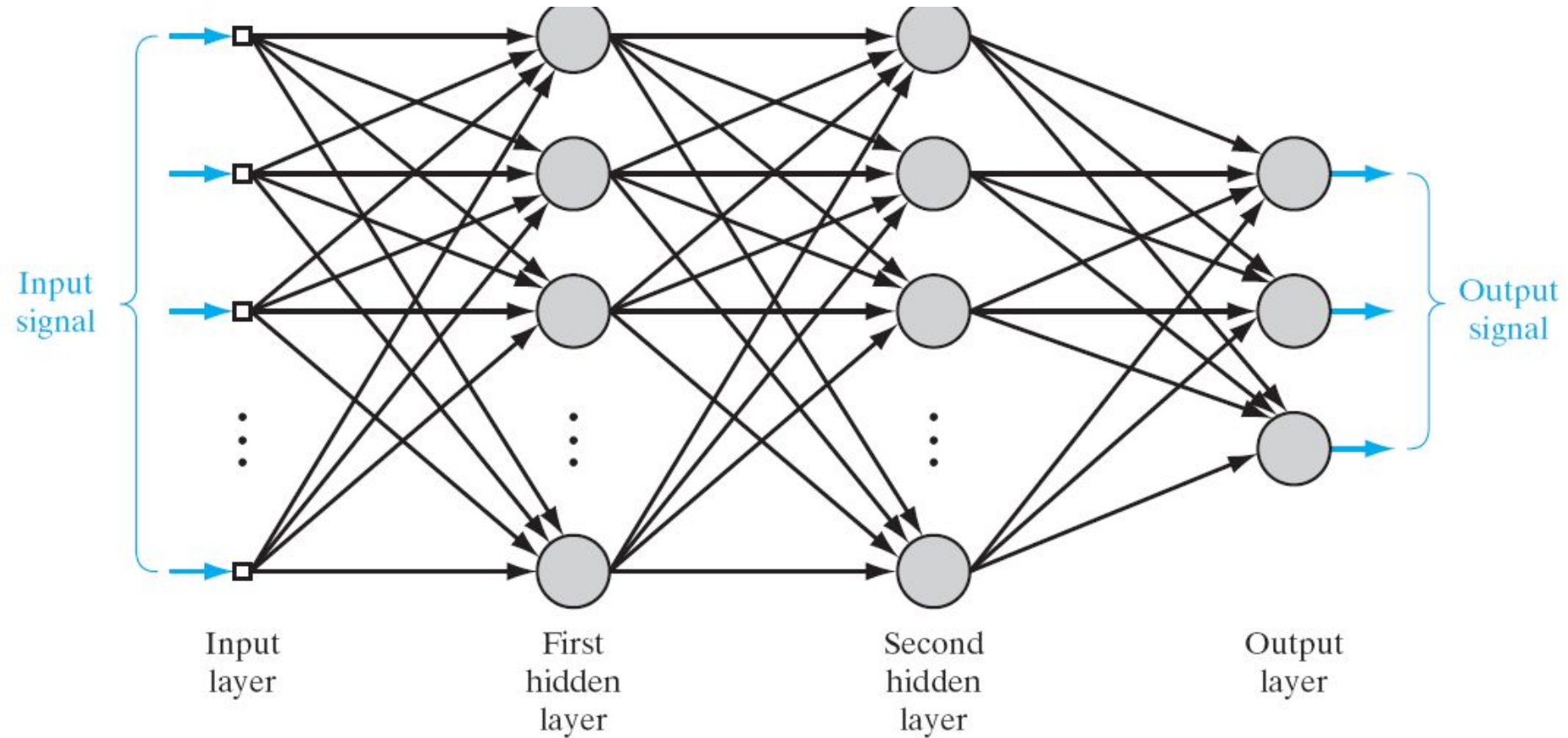


Today's Lecture

ANN: Multi-Layer Perceptron

- Need a **complex model to understand complex patterns**, such as object detection, face recognition, etc.
- Most commonly used architecture is known as the **multilayer perceptron**.
- Basic features of MLP
 - A differentiable **nonlinear activation function**
 - One or more hidden layers
 - High degree of connectivity (fully connected)

Artificial Neural Network



Artificial Neural Network

- A popular method for the training of multilayer perceptron is the **back-propagation algorithm (Generalized Delta Rule)**
- *Forward phase:*
 - The input data is **propagated through the network**, layer by layer, until it reaches the output.
- *Backward phase:*
 - An **error is calculated** by comparing the output of the network with a desired response.
 - This **error is propagated** through the network in **backward direction**.

Neural Network

- Network **architecture or topology** plays an important role for neural net classification.
 - **Optimal topology** for the problem at hand.
- Empirically find the network architectures
 - The number of **hidden layers, units, feedback connections**, and so on.
- **Selecting models** (network topologies) and **estimating parameters** (training via backpropagation) enable you to try out **alternate models**.

Neural Network

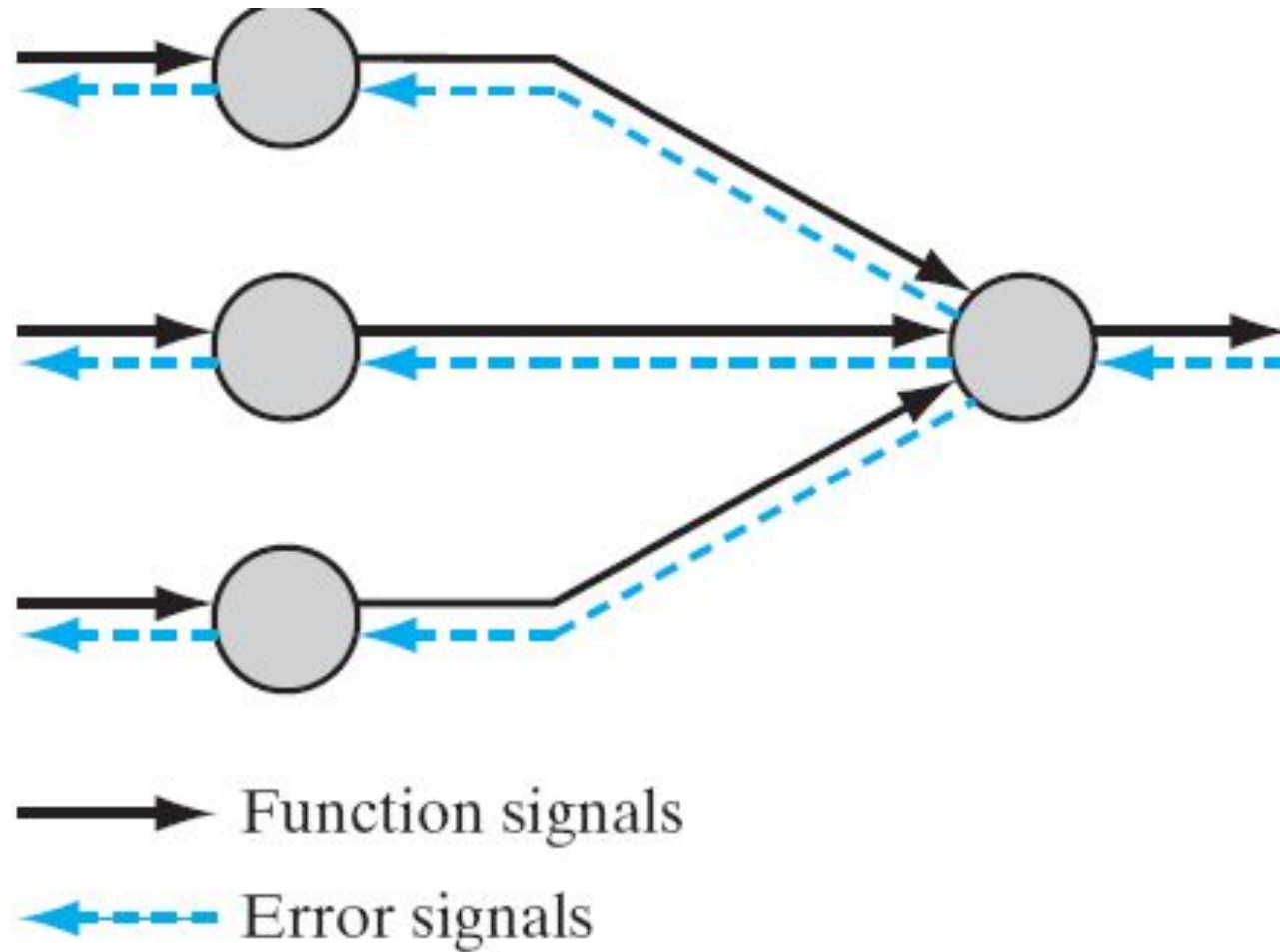
- Adjustments must be made to the weights of the network.
- Calculation of the adjustments for the output layer is straightforward
- It is more challenging for the hidden layers.

Neural Network

- **Hidden Neurons**

- The hidden neurons act as *feature detectors*
- The hidden neurons gradually “discover” the main features to understand the training data.
- They perform nonlinear transformations on the input
 - *Feature space.*
- Classes may be more easily separated from each other

Neural Network



Input of Unit J

$$I_j = \sum_i w_{ij} O_i + \theta_j,$$

- w_{ij} is the weight of the connection from unit i in the previous layer to unit j
- O_i is the output of unit i from the previous layer
- θ_j is the **bias** of the unit

Output of Unit J

$$O_j = \frac{1}{1 + e^{-I_j}}.$$

Back propagation of Error

For the **Output** layer:

$$Err_j = O_j(1 - O_j)(T_j - O_j),$$

For the **Hidden** layers:

$$Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk},$$

Update Weights and Bias

Update Weights:

$$\Delta w_{ij} = (l) Err_j O_i.$$

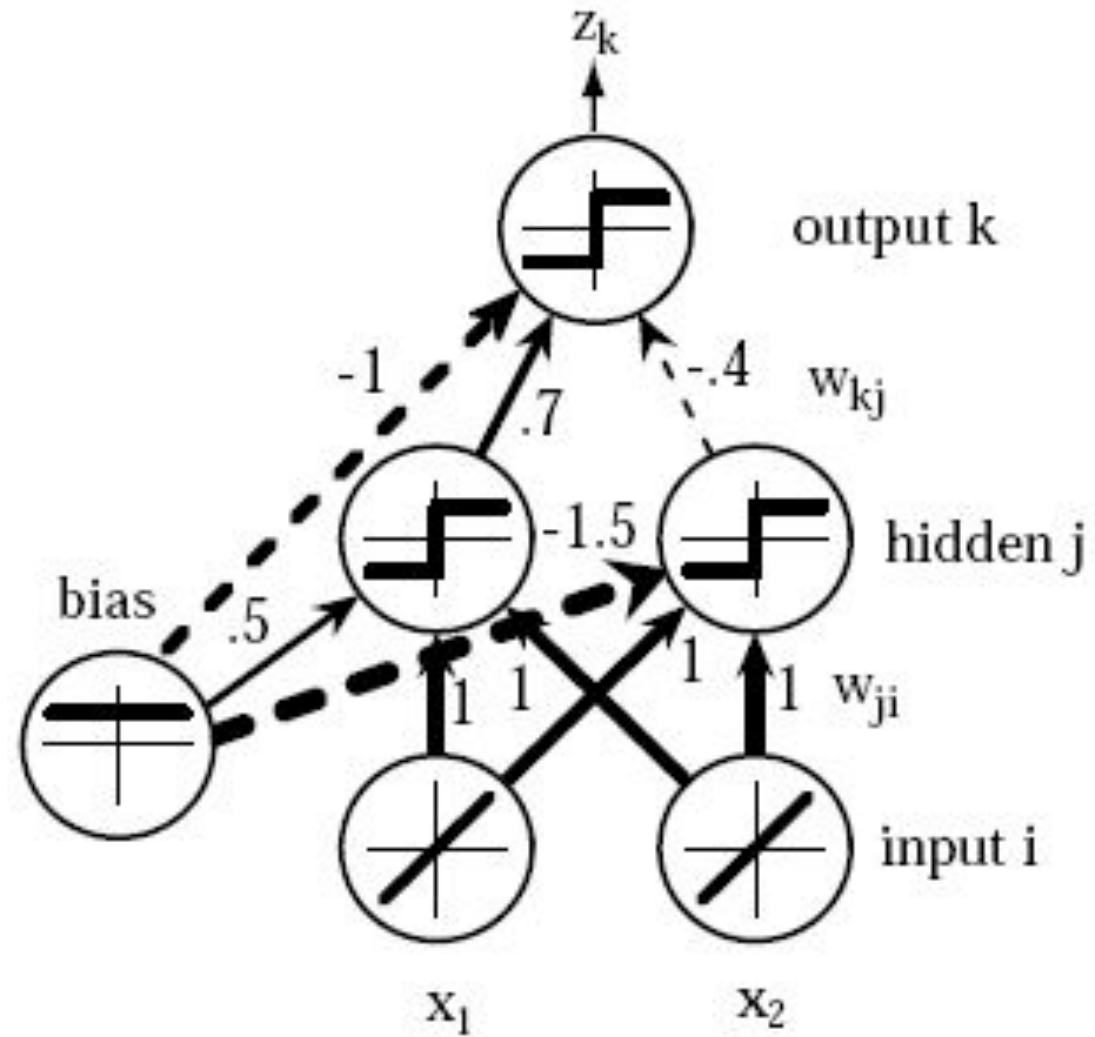
$$w_{ij} = w_{ij} + \Delta w_{ij}.$$

Update Bias:

$$\Delta \theta_j = (l) Err_j.$$

$$\theta_j = \theta_j + \Delta \theta_j.$$

Neural Network



Neural Network

- Each hidden unit performs the **weighted sum of its inputs** to form its (scalar) *net activation or net*.

$$net_j = \sum_{i=1}^d x_i w_{ji} + w_{j0} = \sum_{i=0}^d x_i w_{ji} \equiv \mathbf{w}_j^t \mathbf{x}$$

- The subscript i indices' units on the input layer,
- j for the hidden;
- w_{ji} denotes the input-to-hidden layer weights at the hidden unit j .

Neural Network

- Each **hidden unit produces an output** which is a **nonlinear function** of its activation, $f(net)$

$$y_j = f(\text{net}_j)$$

- $f(x)$ is an **activation function**.
 - Many choices
 - Also called the transfer function or “nonlinearity” of a unit
 - Example, Sigmoid function

$$f(net) = \frac{1}{1 + e^{-Wx}}$$

Neural Network

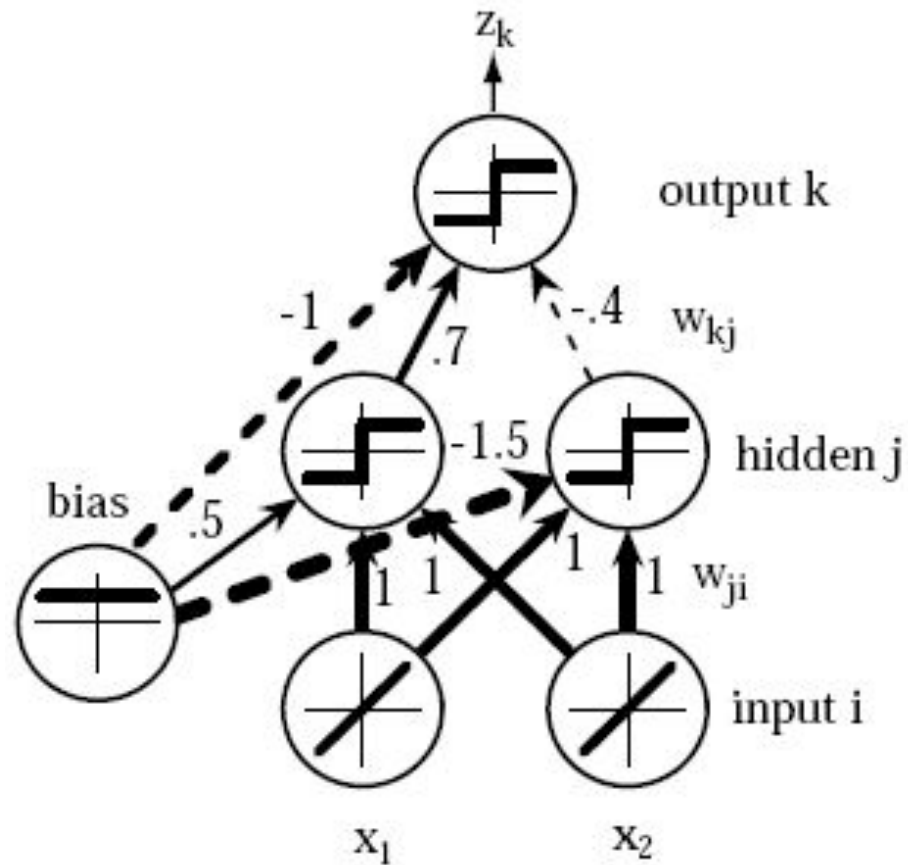
- Each output unit (neuron) similarly computes its net activation based on the hidden unit signals as:

$$net_k = \sum_{j=1}^{n_H} y_j w_{kj} + w_{k0} = \sum_{j=0}^{n_H} y_j w_{kj} = \mathbf{w}_k^t \mathbf{y}$$

- Each **output unit** then **computes the nonlinear** function of its **net**

$$z_k = f(\text{net}_k)$$

Neural Network



Inputs		Output
x_1	x_2	x
0	0	0
0	1	1
1	0	1
1	1	0

Neural Network

- For a three layer network, we can generalize the formula to get the final output as

$$g_k(x) \equiv z_k = f \left(\sum_{j=1}^{n_H} w_{kj} f \left(\sum_{i=1}^d w_{ji} x_i + w_{j0} \right) + w_{k0} \right)$$

- Can we put different activation functions at the output layer to differ from those in the hidden layer?
 - Yes 😊

Backpropagation algorithm

- We can calculate the desired error for the output units as:

$$J(\mathbf{w}) \equiv 1/2 \sum_{k=1}^c (t_k - z_k)^2 = 1/2(\mathbf{t} - \mathbf{z})^2$$

- The **backpropagation learning rule is based on gradient descent**.
- The weights are initialized with random values, and are changed in a direction that will reduce the error:

$$\Delta \mathbf{w} = -\eta \frac{\partial J}{\partial \mathbf{w}}$$

- where η is the **learning rate**

Backpropagation algorithm

- The weight vector will be updated in iterative manner as:

$$w(m + 1) = w(m) + \Delta w(m)$$

$$w(m + 1) = w(m) - \eta \frac{\partial J}{\partial w}$$

- The weights throughout the network can be updated using.

$$\Delta w_{mn} = -\eta \frac{\partial J}{\partial w_{mn}}$$

- Now the challenge is to update the weights
 - Some are **not explicitly dependent on incoming weights**

Backpropagation algorithm

-

Backpropagation algorithm

-

Backpropagation algorithm

$$\begin{aligned}\frac{\partial J}{\partial y_j} &= \frac{\partial}{\partial y_j} \left[1/2 \sum_{k=1}^c (t_k - z_k)^2 \right] \\ &= - \sum_{k=1}^c (t_k - z_k) \frac{\partial z_k}{\partial y_j} \\ &= - \sum_{k=1}^c (t_k - z_k) \frac{\partial z_k}{\partial net_k} \frac{\partial net_k}{\partial y_j} \\ &= - \sum_{k=1}^c (t_k - z_k) f'(net_k) w_{jk}.\end{aligned}$$

This last equation expresses how the hidden unit output, y_j , affects the error at each output unit.

Backpropagation algorithm

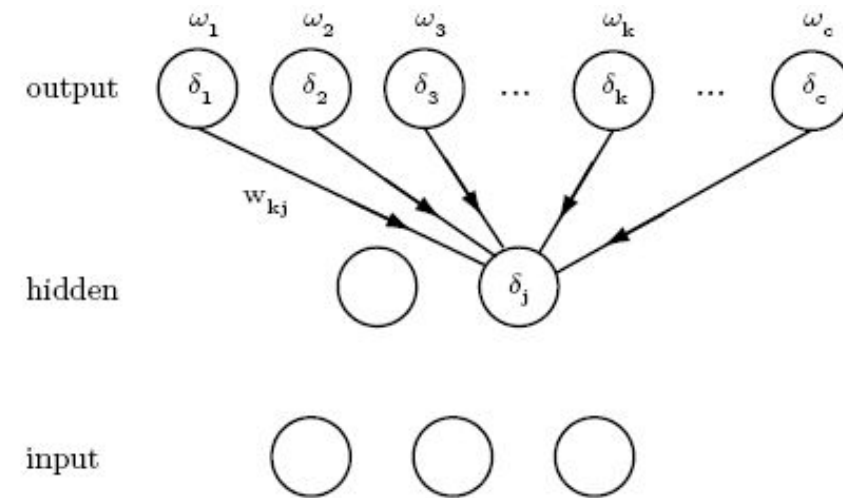
- Similar to the sensitivity we calculated for the output units, we can also calculate the sensitivity for hidden units similar to previous equation as:

$$\delta_j \equiv f'(net_j) \sum_{k=1}^c w_{kj} \delta_k.$$

- This equation indicates that the sensitivity at a hidden unit is simply the sum of the individual sensitivities at the output units weighted by the hidden-to-output weights w_{jk} , all multiplied by $f'(net_j)$.

Backpropagation algorithm

- The sensitivity at a hidden unit is proportional to the **weighted sum of the sensitivities at the output units**.
- The output unit sensitivities are thus **propagated “back” to the hidden units**.



Backpropagation algorithm

-

activations

x

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial x}$$

“local gradient”

$$\frac{\partial z}{\partial x}$$

$$\frac{\partial z}{\partial y}$$

f

z

$$\frac{\partial L}{\partial z}$$

gradients

y

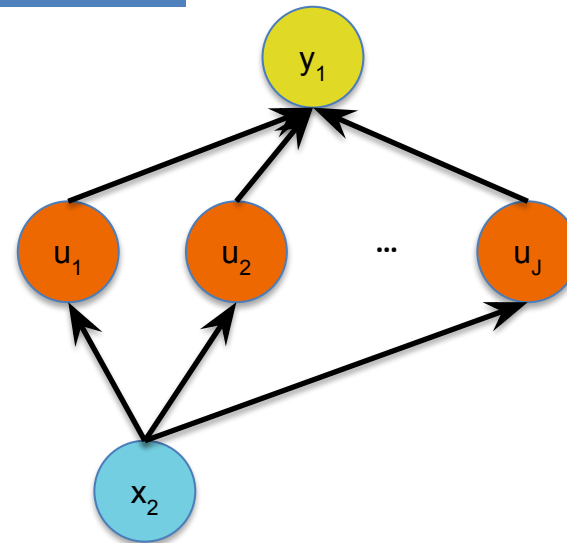
$$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial y}$$

Chain Rule

Given: $y = g(u)$ and $u = h(x)$

Chain Rule:

$$\frac{dy_i}{dx_k} = \sum_{j=1}^J \frac{dy_i}{du_j} \frac{du_j}{dx_k}, \quad \forall i, k$$



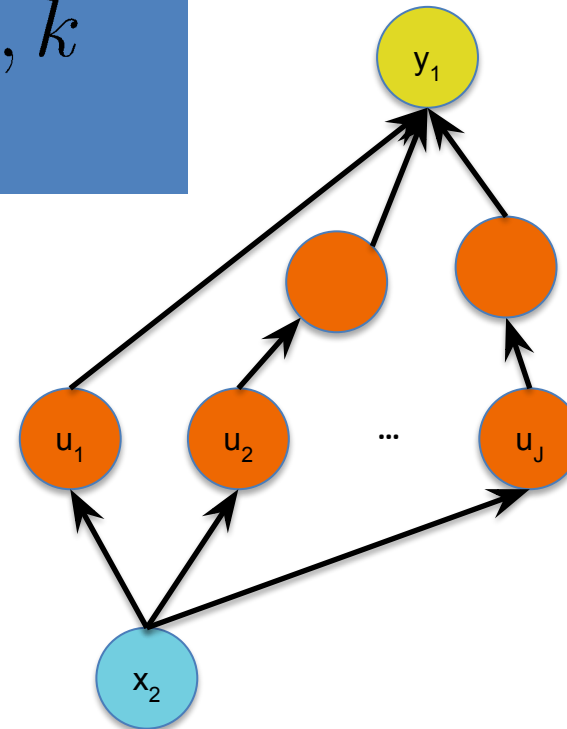
Chain Rule

Given: $y = g(u)$ and $u = h(x)$

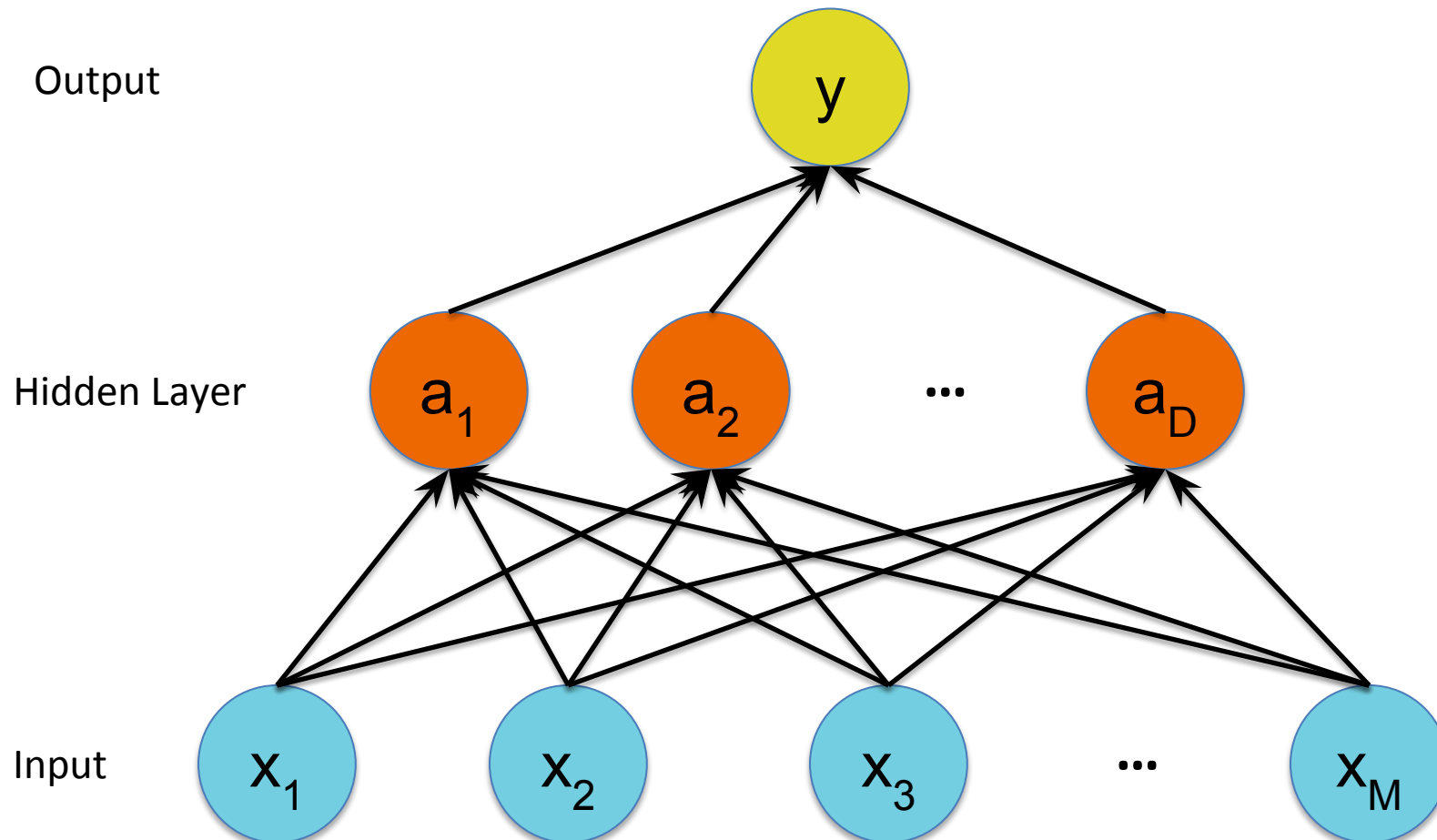
Chain Rule:

$$\frac{dy_i}{dx_k} = \sum_{j=1}^J \frac{dy_i}{du_j} \frac{du_j}{dx_k}, \quad \forall i, k$$

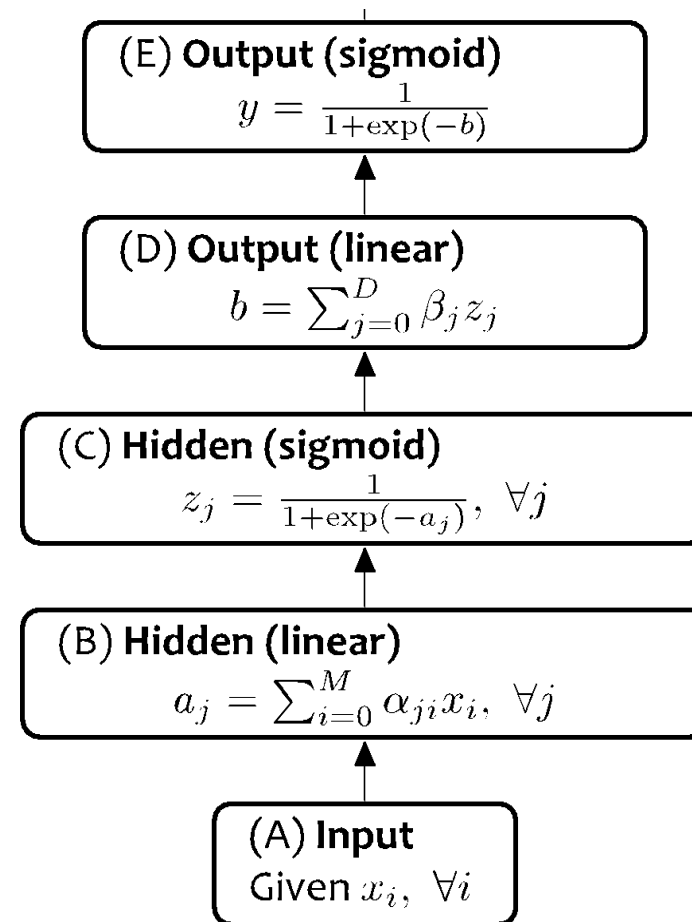
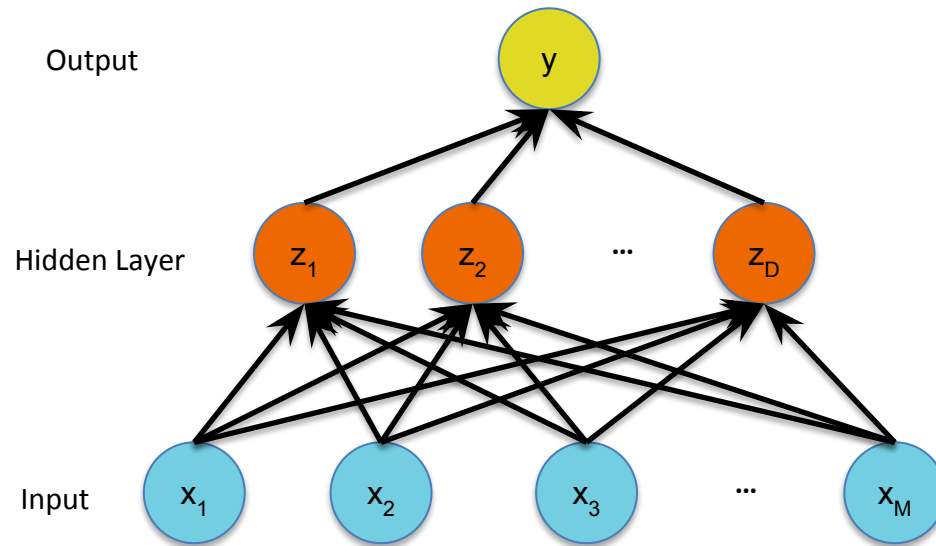
Backpropagation
is just repeated
application of the
chain rule from
Calculus.



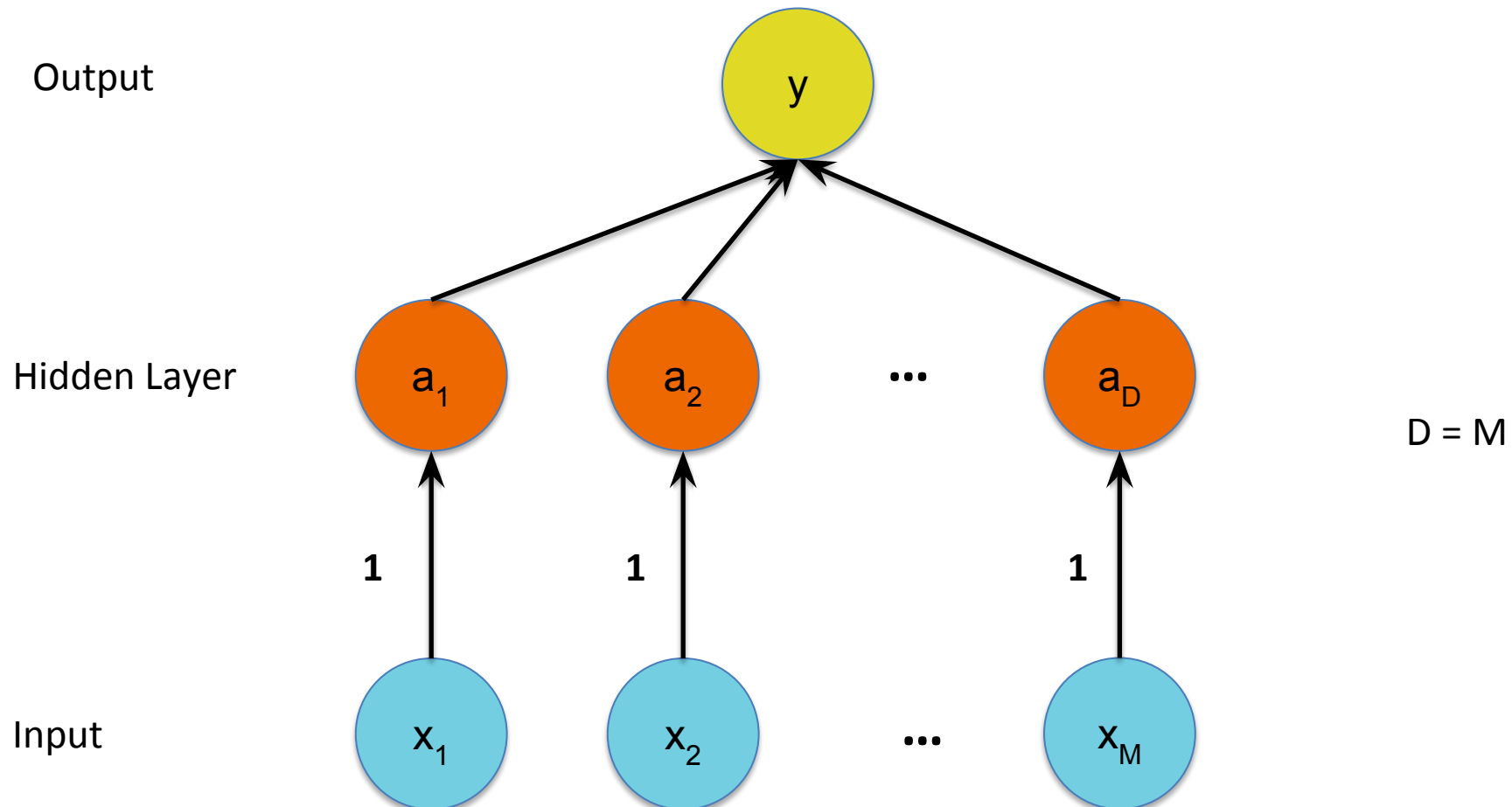
Neural Network



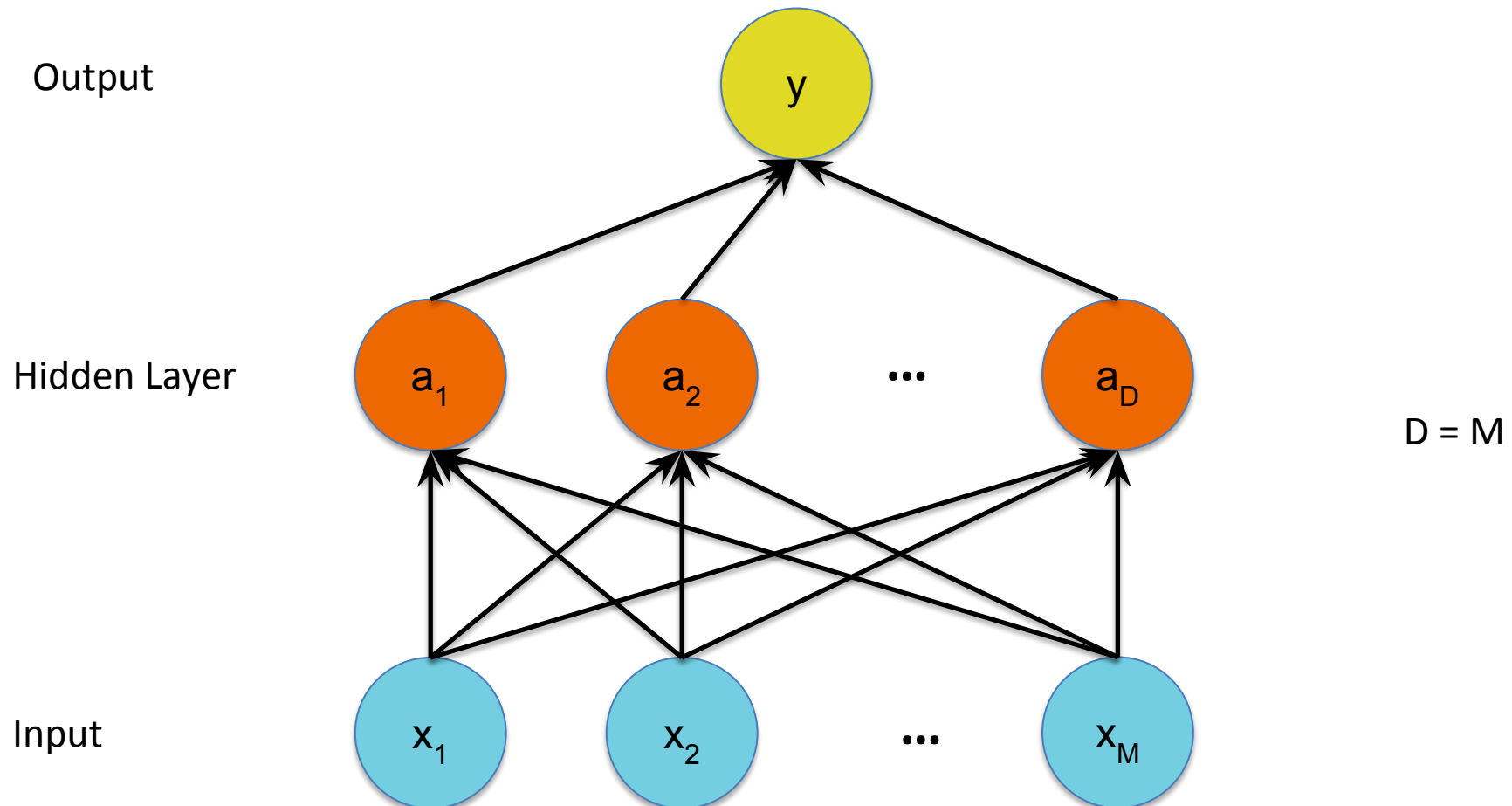
Neural Network



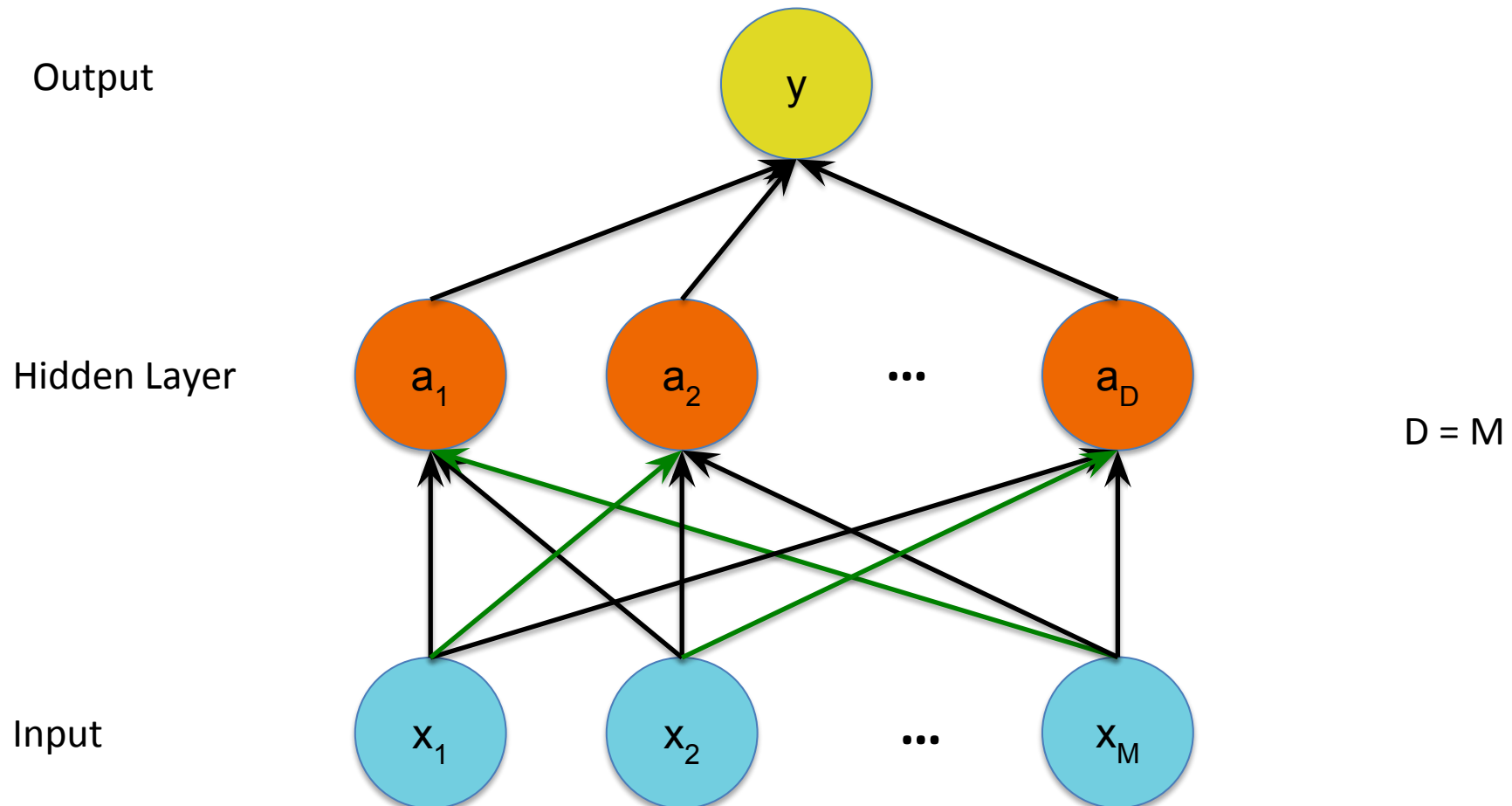
Building a Neural Net



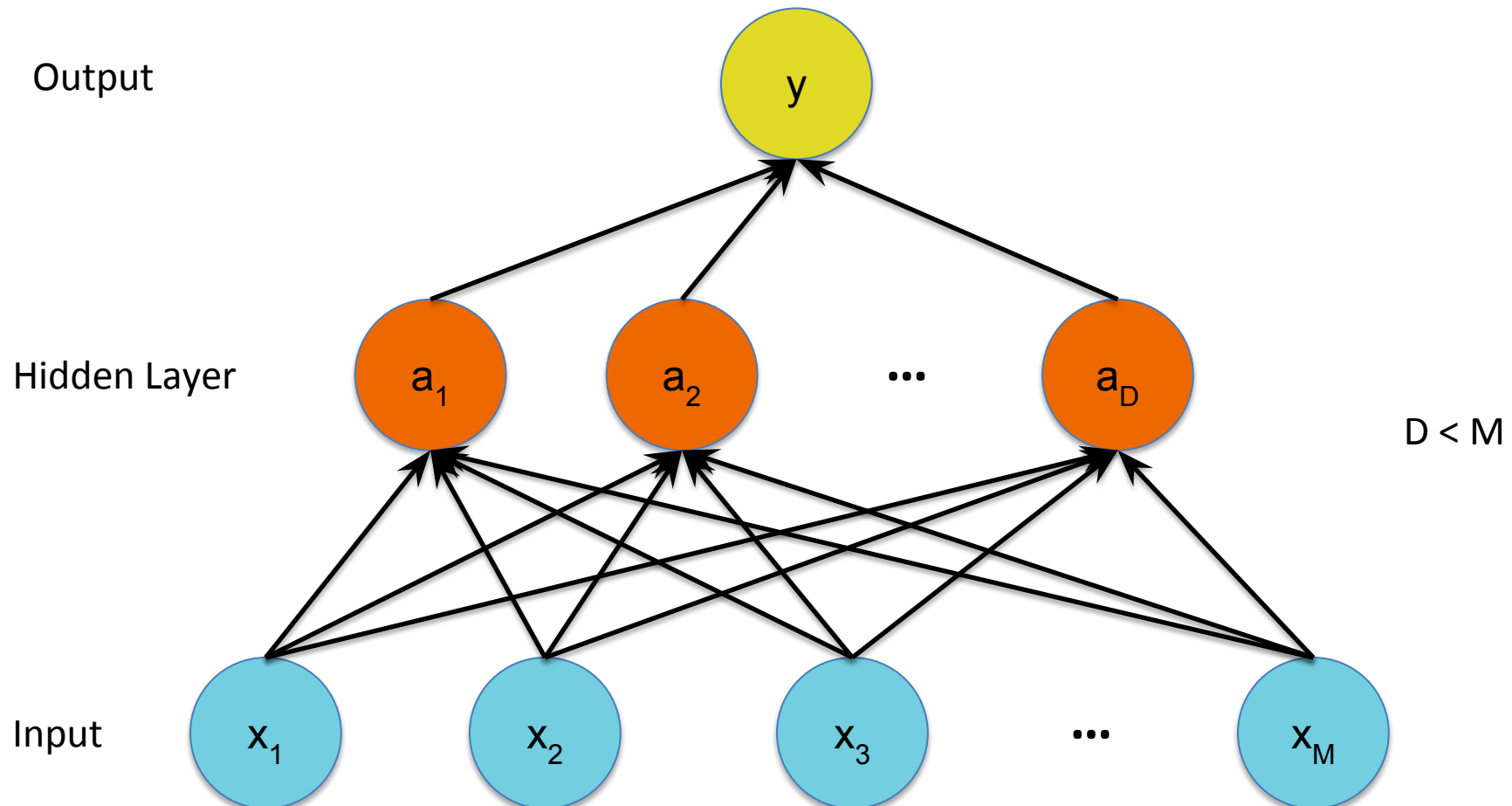
Building a Neural Net



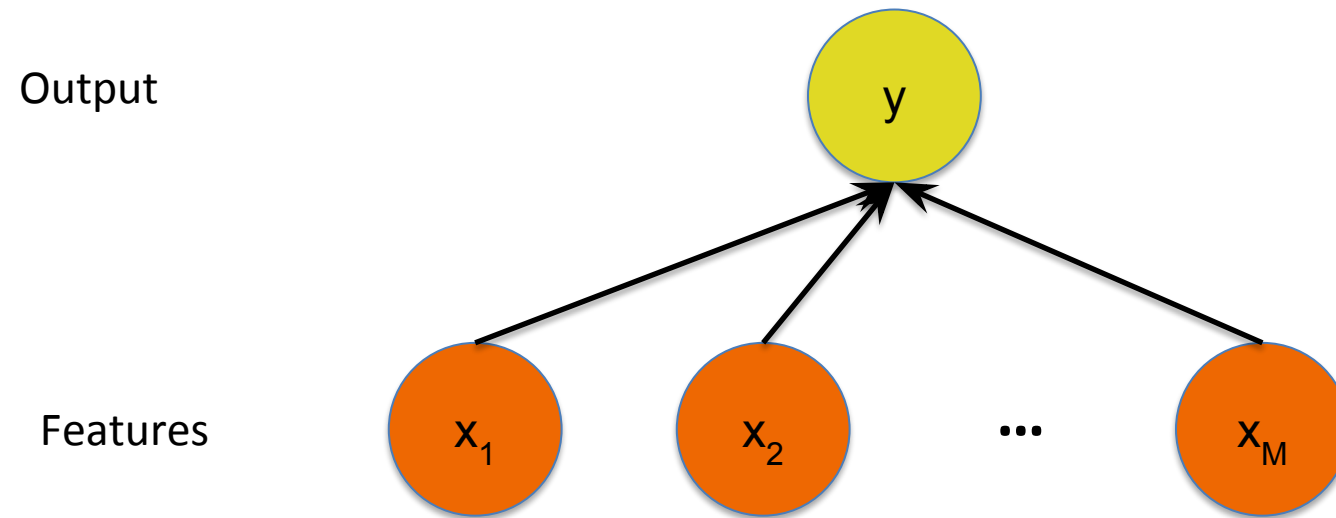
Building a Neural Net



Building a Neural Net

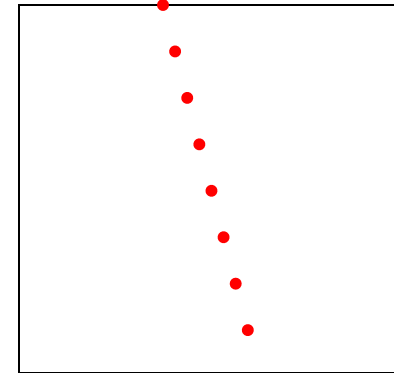
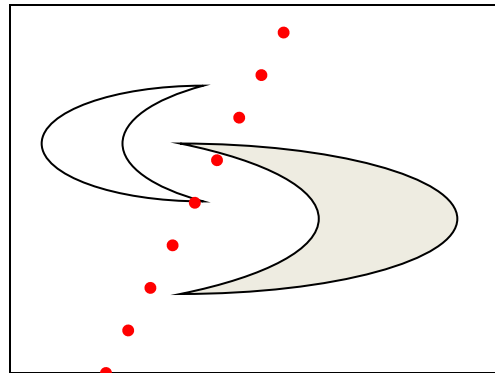
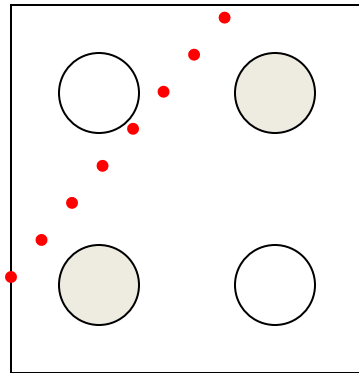
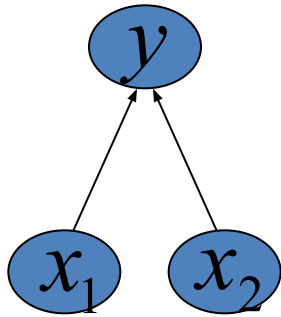


Building a Neural Net



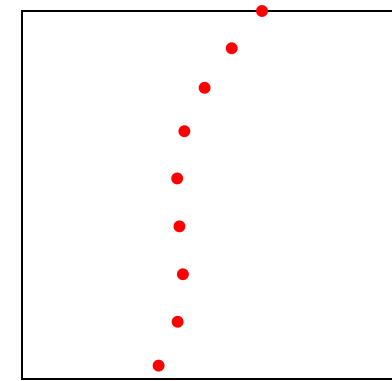
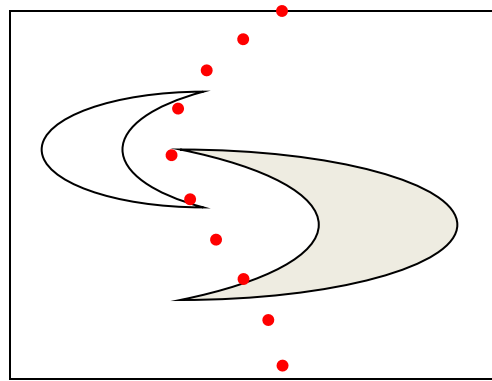
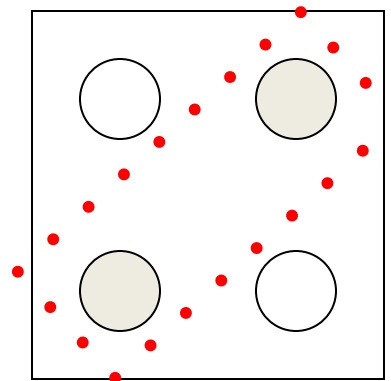
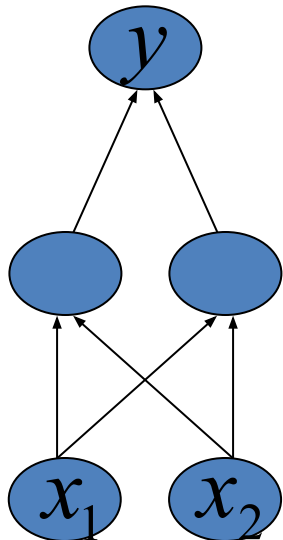
Decision Boundary

- 0 hidden layers: linear classifier
 - Hyperplanes

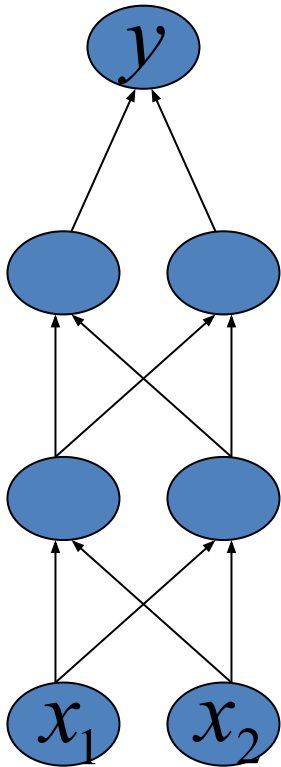


Decision Boundary

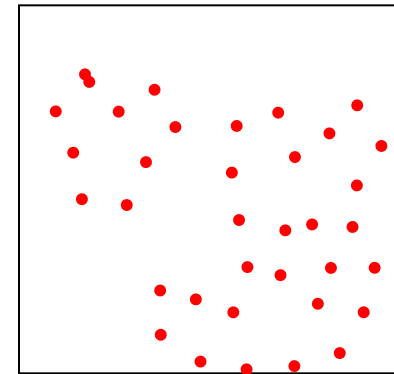
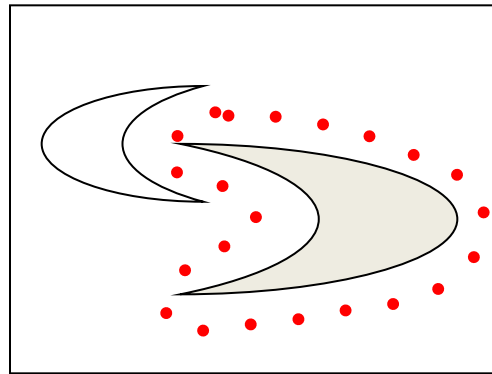
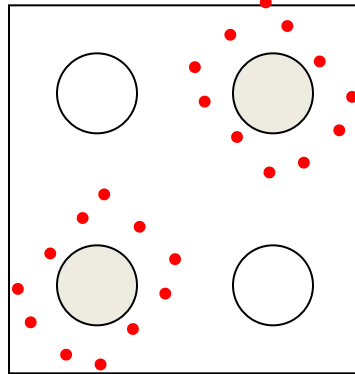
- 1 hidden layer
 - Boundary of convex region (open or closed)



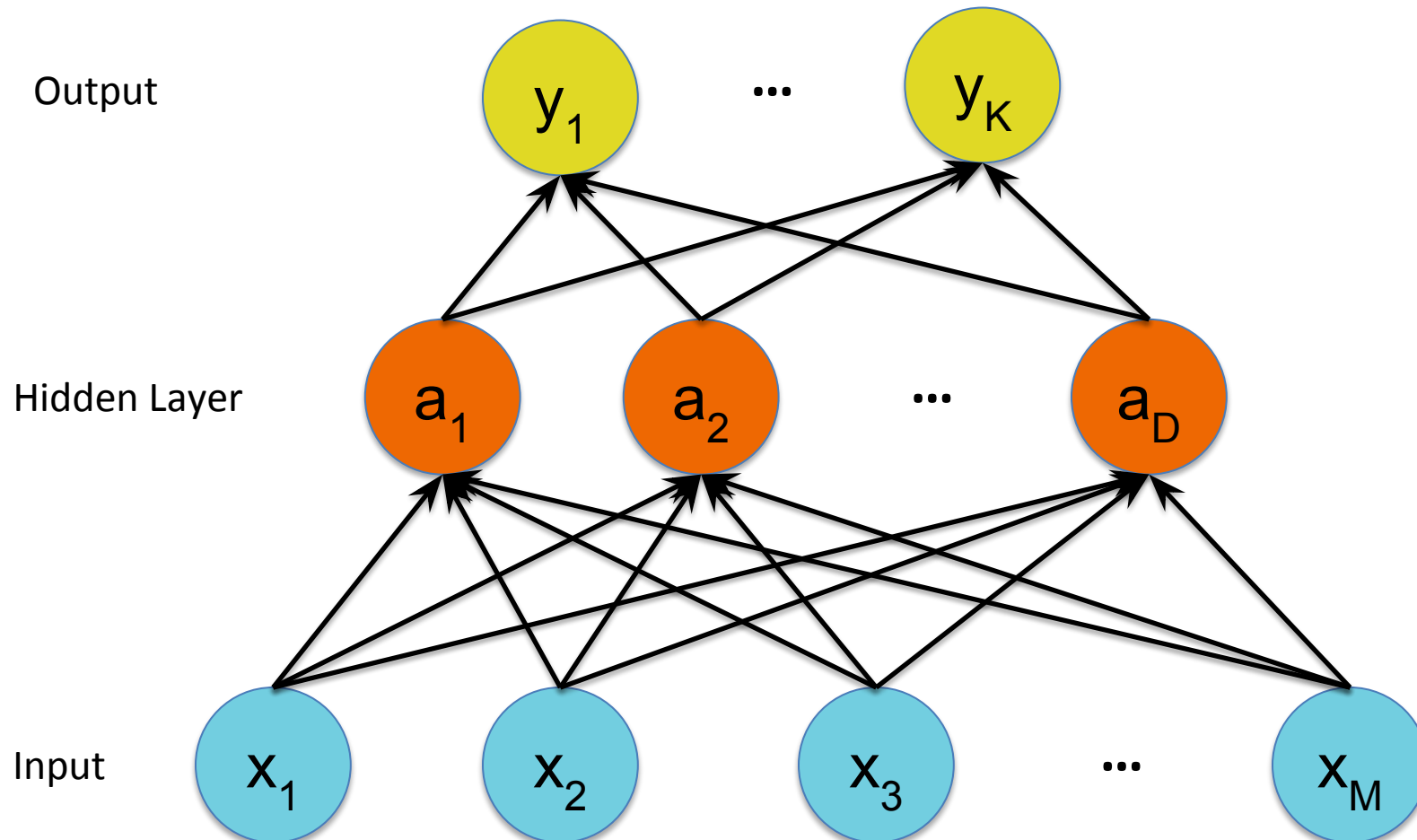
Decision Boundary



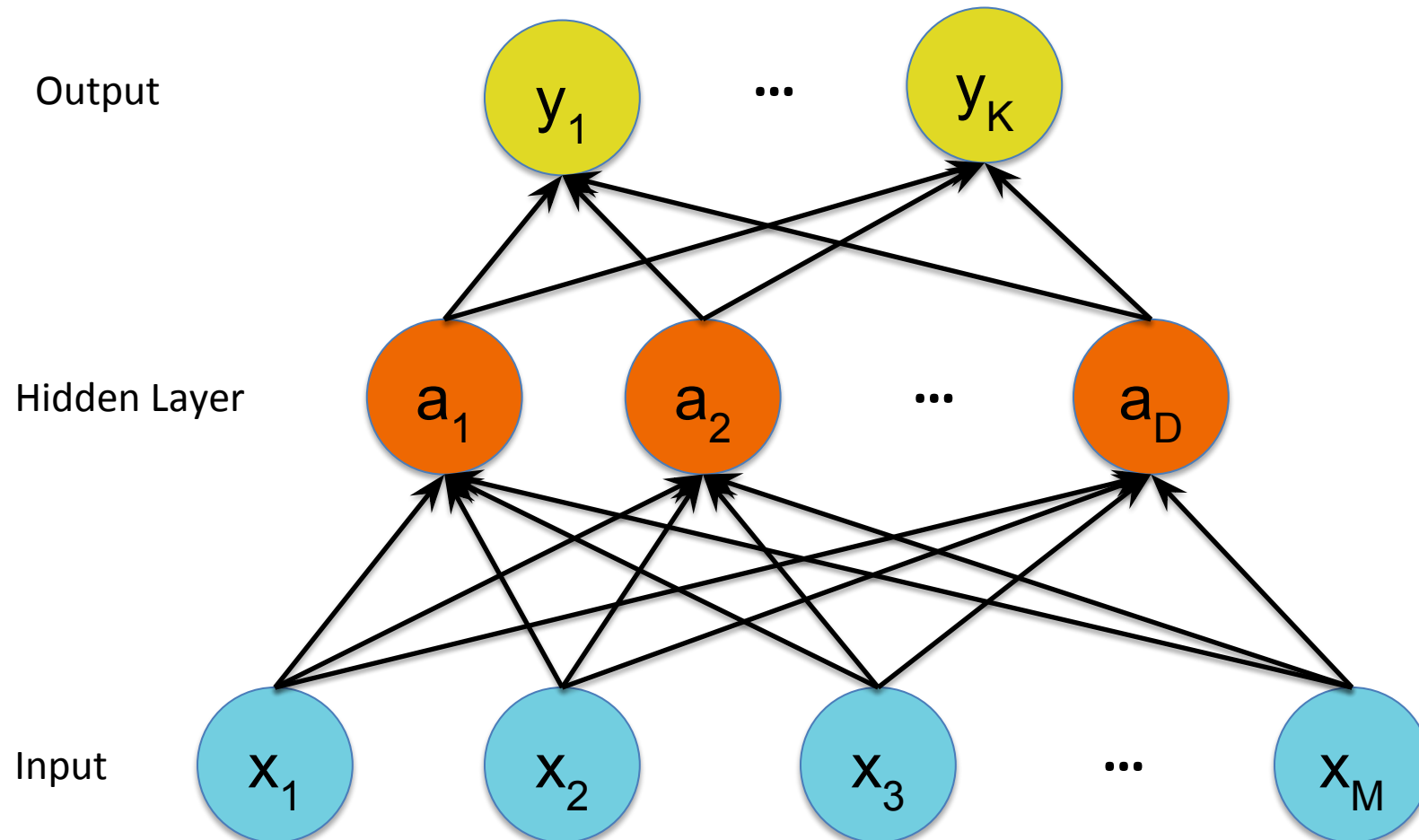
- 2 hidden layers
 - Combinations of convex regions



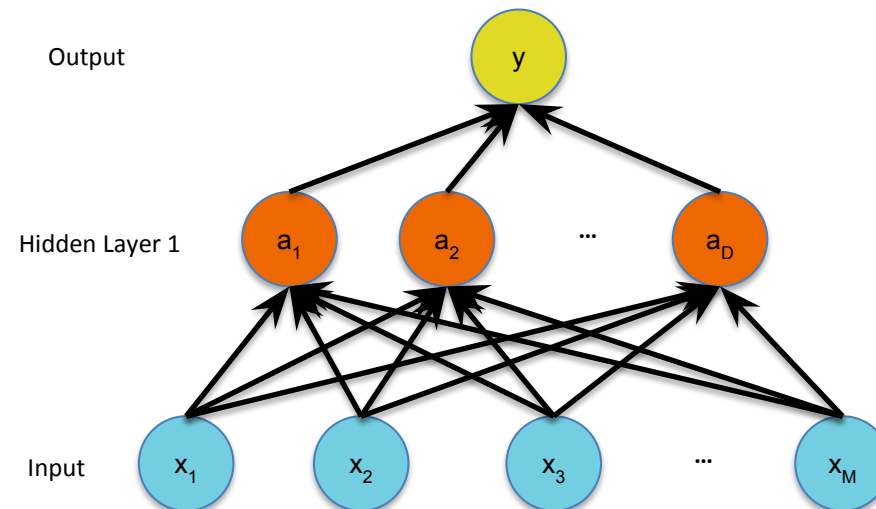
Multi-Class Output



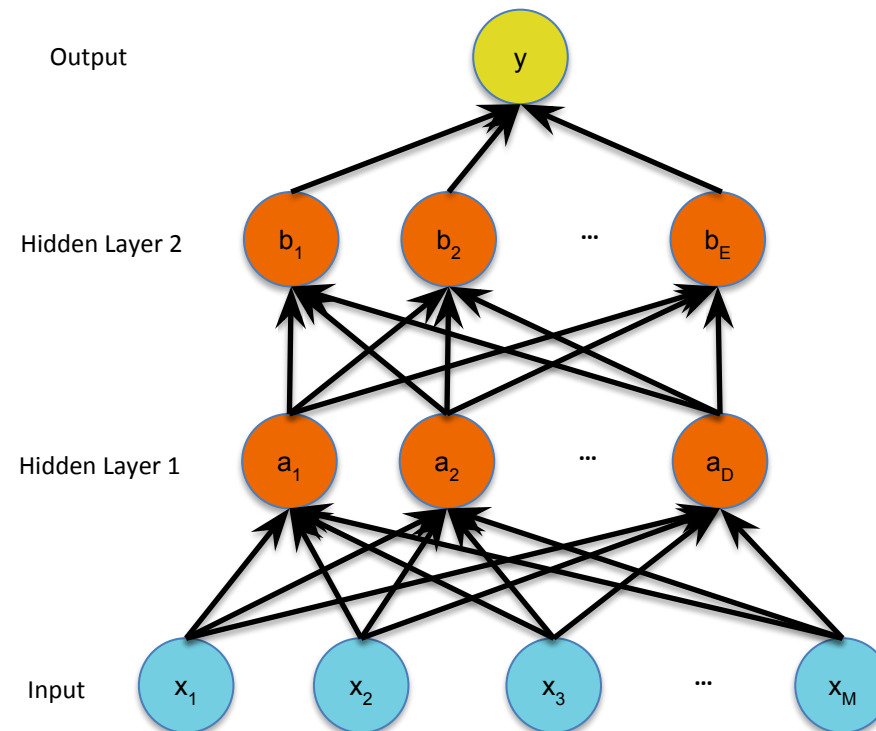
Multi-Class Output



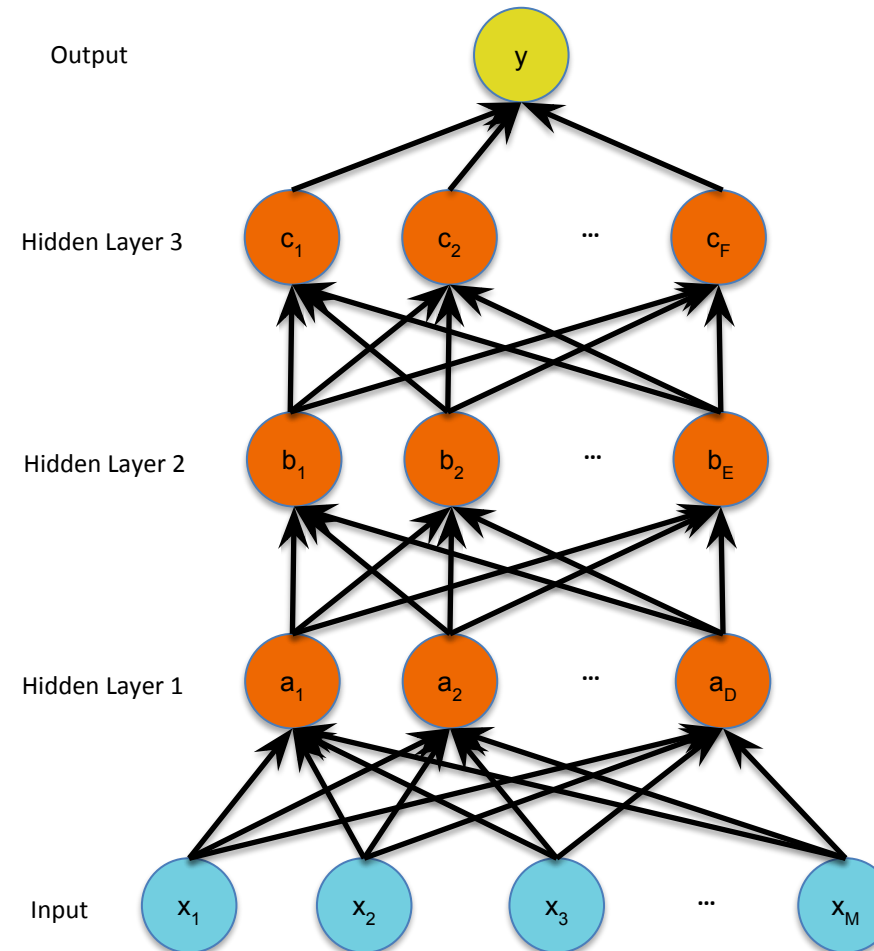
Deeper Networks



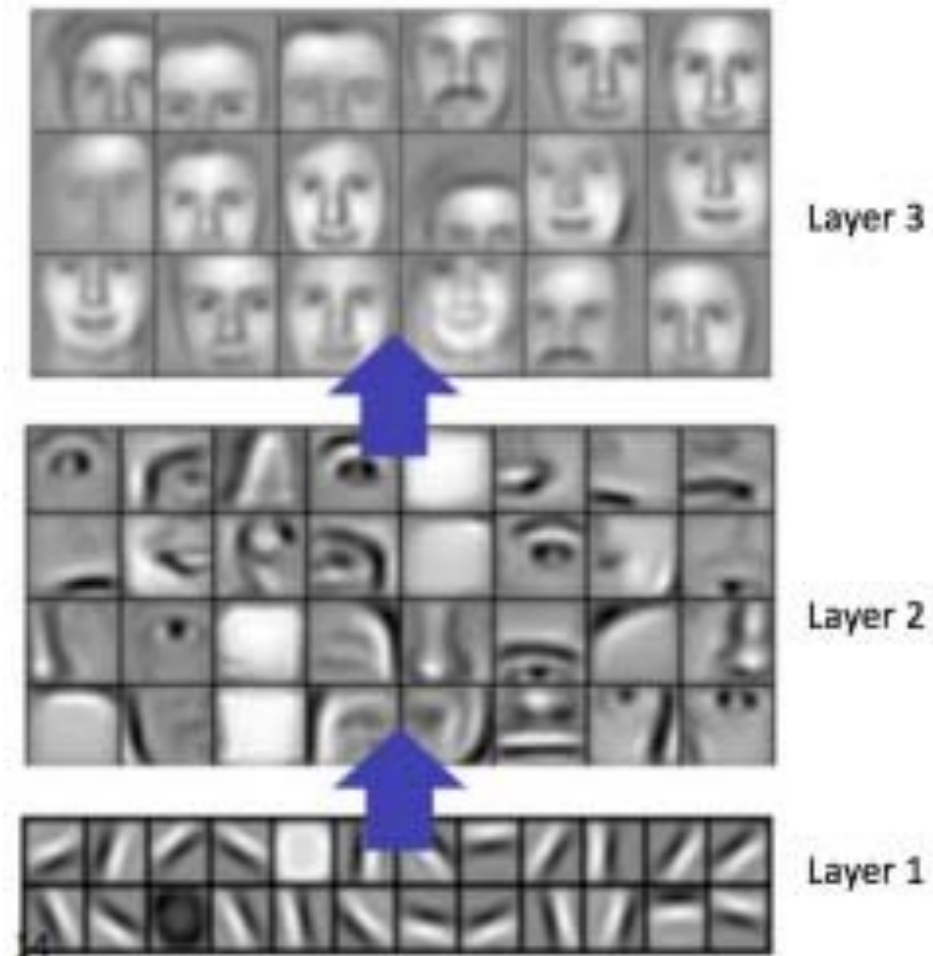
Deeper Networks



Deeper Networks



Deeper Networks



Example

Input values

$X_1=0.05$

$X_2=0.10$

Initial weight

$W_1=0.15$ $w_5=0.40$

$W_2=0.20$ $w_6=0.45$

$W_3=0.25$ $w_7=0.50$

$W_4=0.30$ $w_8=0.55$

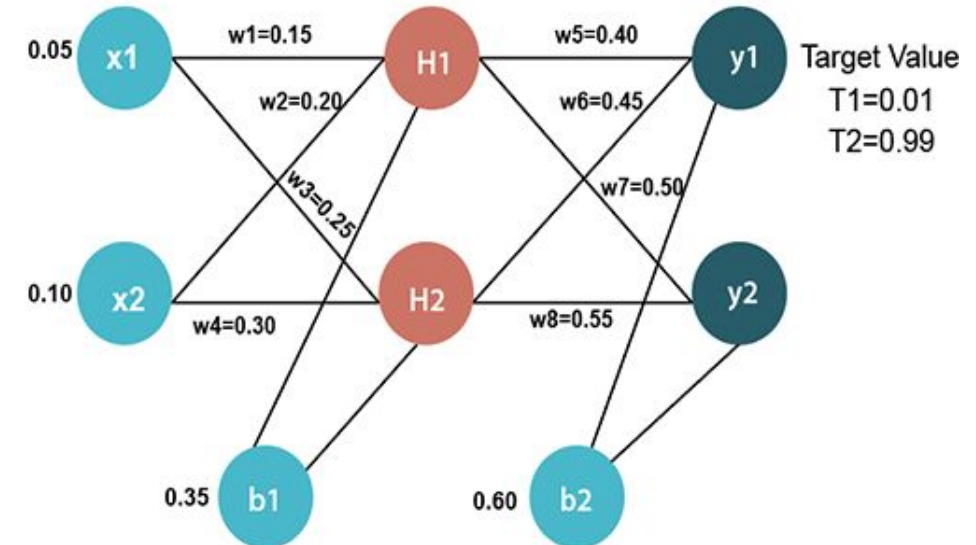
Bias Values

$b_1=0.35$ $b_2=0.60$

Target Values

$T_1=0.01$

$T_2=0.99$



Forward Pass

To find the value of H1 we first multiply the input value from the weights as

$$H1 = x1 \times w_1 + x2 \times w_2 + b1$$

$$H1 = 0.05 \times 0.15 + 0.10 \times 0.20 + 0.35$$

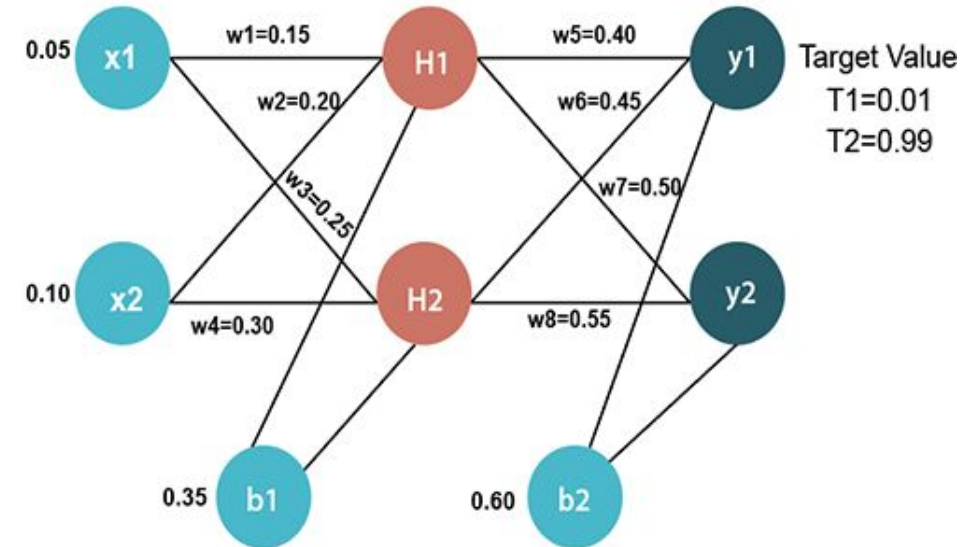
$$\mathbf{H1 = 0.3775}$$

To calculate the final result of H1, we performed the sigmoid function as

$$H1_{final} = \frac{1}{1 + \frac{1}{e^{H1}}}$$

$$H1_{final} = \frac{1}{1 + \frac{1}{e^{0.3775}}}$$

$$\mathbf{H1_{final} = 0.593269992}$$



$$H2 = x1 \times w_3 + x2 \times w_4 + b1$$

$$H2 = 0.05 \times 0.25 + 0.10 \times 0.30 + 0.35$$

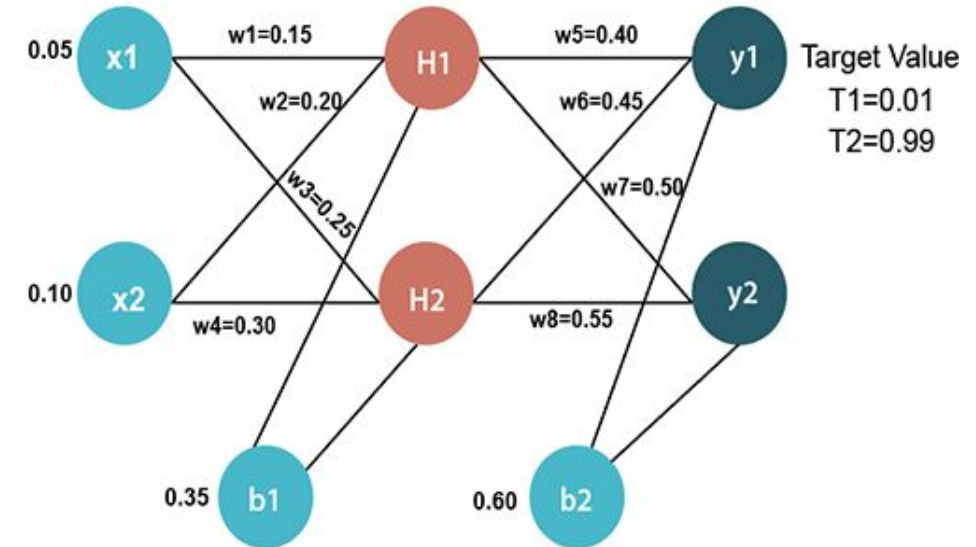
$$\mathbf{H2 = 0.3925}$$

To calculate the final result of H1, we performed the sigmoid function as

$$H2_{final} = \frac{1}{1 + \frac{1}{e^{H2}}}$$

$$H2_{final} = \frac{1}{1 + \frac{1}{e^{0.3925}}}$$

$$\mathbf{H2_{final} = 0.596884378}$$



$$y1 = H1 \times w_5 + H2 \times w_6 + b2$$

$$y1 = 0.593269992 \times 0.40 + 0.596884378 \times 0.45 + 0.60$$

$$\mathbf{y1 = 1.10590597}$$

To calculate the final result of y1 we performed the sigmoid function as

$$y1_{\text{final}} = \frac{1}{1 + \frac{1}{e^{y1}}}$$

$$y1_{\text{final}} = \frac{1}{1 + \frac{1}{e^{1.10590597}}}$$

$$\mathbf{y1_{\text{final}} = 0.75136507}$$

We will calculate the value of y2 in the same way as y1

$$y2 = H1 \times w_7 + H2 \times w_8 + b2$$

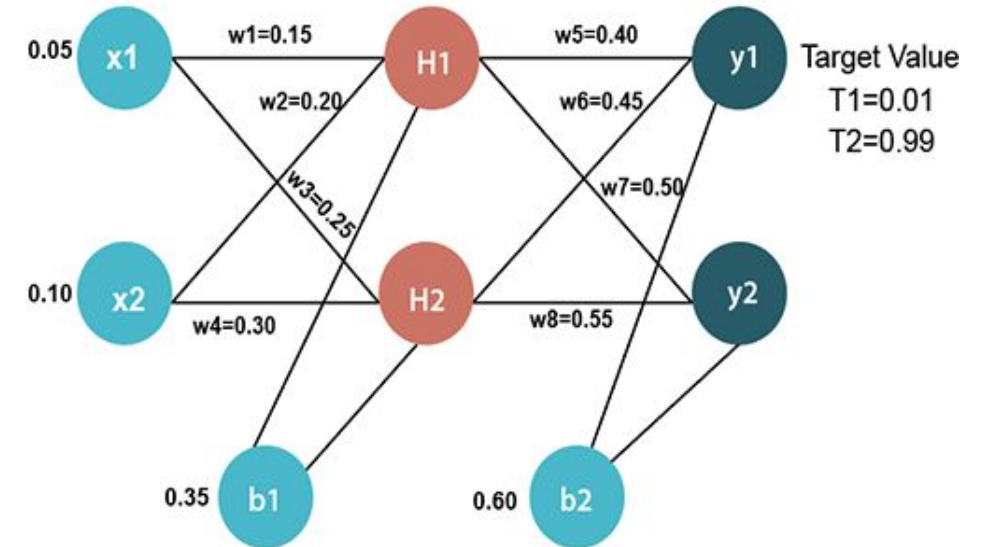
$$y2 = 0.593269992 \times 0.50 + 0.596884378 \times 0.55 + 0.60$$

$$\mathbf{y2 = 1.2249214}$$

$$y2_{\text{final}} = \frac{1}{1 + \frac{1}{e^{y2}}}$$

$$y2_{\text{final}} = \frac{1}{1 + \frac{1}{e^{1.2249214}}}$$

$$\mathbf{y2_{\text{final}} = 0.772928465}$$

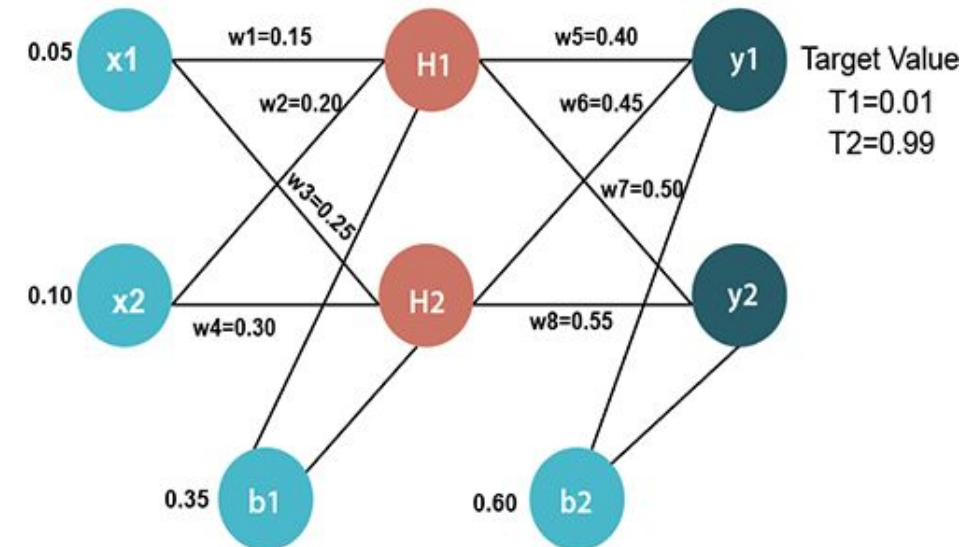


Example

$$E_{\text{total}} = \sum \frac{1}{2} (\text{target} - \text{output})^2$$

So, the total error is

$$\begin{aligned} &= \frac{1}{2} (t1 - y1_{\text{final}})^2 + \frac{1}{2} (T2 - y2_{\text{final}})^2 \\ &= \frac{1}{2} (0.01 - 0.75136507)^2 + \frac{1}{2} (0.99 - 0.772928465)^2 \\ &= 0.274811084 + 0.0235600257 \\ &\mathbf{E_{\text{total}} = 0.29837111} \end{aligned}$$



Example

Backward pass at the output layer

To update the weight, we calculate the error correspond to each weight

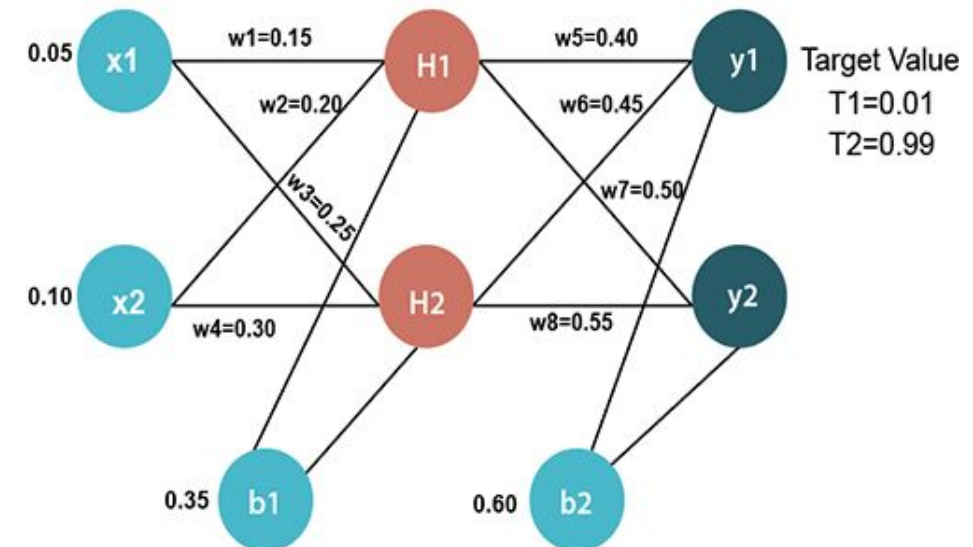
$$\text{Error}_w = \frac{\partial E_{\text{total}}}{\partial w}$$

We perform backward process so first consider the last weight w_5 as

$$\text{Error}_{w_5} = \frac{\partial E_{\text{total}}}{\partial w_5} \dots \dots \dots (1)$$

$$E_{\text{total}} = \frac{1}{2} (T1 - y1_{\text{final}})^2 + \frac{1}{2} (T2 - y2_{\text{final}})^2 \dots \dots \dots (2)$$

$$\frac{\partial E_{\text{total}}}{\partial w_5} = \frac{\partial E_{\text{total}}}{\partial y1_{\text{final}}} \times \frac{\partial y1_{\text{final}}}{\partial y1} \times \frac{\partial y1}{\partial w_5} \dots \dots \dots (3)$$



$$\frac{\partial E_{\text{total}}}{\partial y1_{\text{final}}} = \frac{\partial(\frac{1}{2}(T1 - y1_{\text{final}})^2 + \frac{1}{2}(T2 - y2_{\text{final}})^2)}{\partial y1_{\text{final}}}$$

$$= 2 \times \frac{1}{2} \times (T1 - y1_{\text{final}})^{2-1} \times (-1) + 0$$

$$= -(T1 - y1_{\text{final}})$$

$$= -(0.01 - 0.75136507)$$

$$\frac{\partial E_{\text{total}}}{\partial y1_{\text{final}}} = 0.74136507 \dots \dots \dots (4)$$

$$y1_{\text{final}} = \frac{1}{1 + e^{-y1}} \dots \dots \dots (5)$$

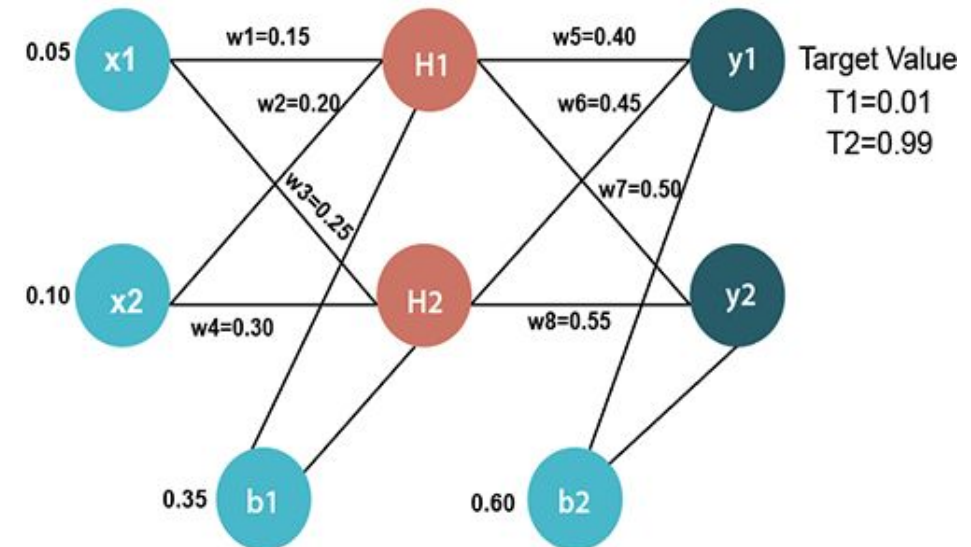
$$\frac{\partial y1_{\text{final}}}{\partial y1} = \frac{\partial(\frac{1}{1 + e^{-y1}})}{\partial y1}$$

$$= \frac{e^{-y1}}{(1 + e^{-y1})^2}$$

$$= e^{-y1} \times (y1_{\text{final}})^2 \dots \dots \dots (6)$$

$$y1_{\text{final}} = \frac{1}{1 + e^{-y1}}$$

$$e^{-y1} = \frac{1 - y1_{\text{final}}}{y1_{\text{final}}} \dots \dots \dots (7)$$



Putting the value of e^{-y} in equation (5)

$$= \frac{1 - y1_{final}}{y1_{final}} \times (y1_{final})^2$$

$$= y1_{final} \times (1 - y1_{final})$$

$$= 0.75136507 \times (1 - 0.75136507)$$

$$\frac{\partial y1_{final}}{\partial y1} = 0.186815602 \dots \dots \dots (8)$$

$$y1 = H1_{final} \times w5 + H2_{final} \times w6 + b2 \dots \dots \dots (9)$$

$$\frac{\partial y1}{\partial w5} = \frac{\partial (H1_{final} \times w5 + H2_{final} \times w6 + b2)}{\partial w5}$$

$$= H1_{final}$$

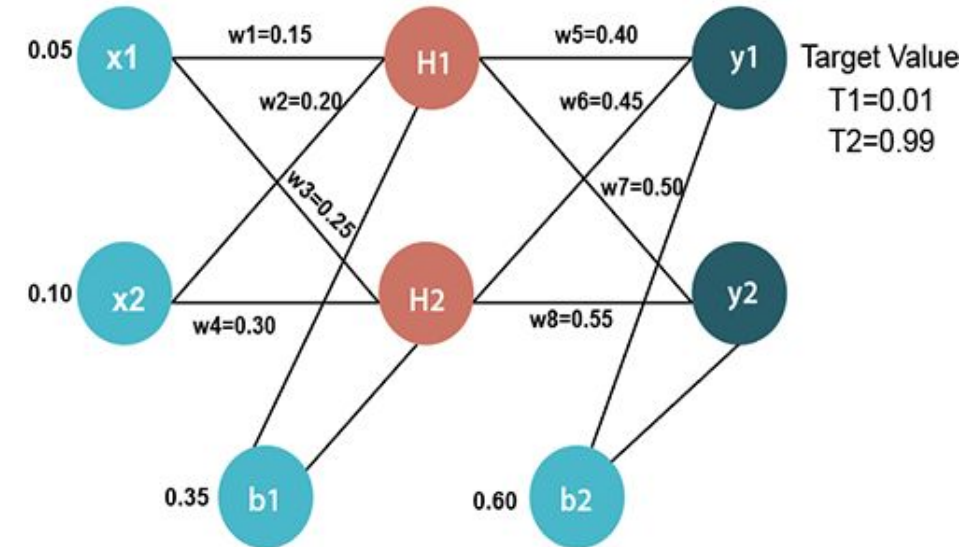
$$\frac{\partial y1}{\partial w5} = 0.596884378 \dots \dots \dots (10)$$

So, we put the values of $\frac{\partial E_{total}}{\partial y1_{final}}$, $\frac{\partial y1_{final}}{\partial y1}$, and $\frac{\partial y1}{\partial w5}$ in equation no (3) to find the final result.

$$\frac{\partial E_{total}}{\partial w5} = \frac{\partial E_{total}}{\partial y1_{final}} \times \frac{\partial y1_{final}}{\partial y1} \times \frac{\partial y1}{\partial w5}$$

$$= 0.74136507 \times 0.186815602 \times 0.593269992$$

$$\text{Error}_{w5} = \frac{\partial E_{total}}{\partial w5} = 0.0821670407 \dots \dots \dots (11)$$



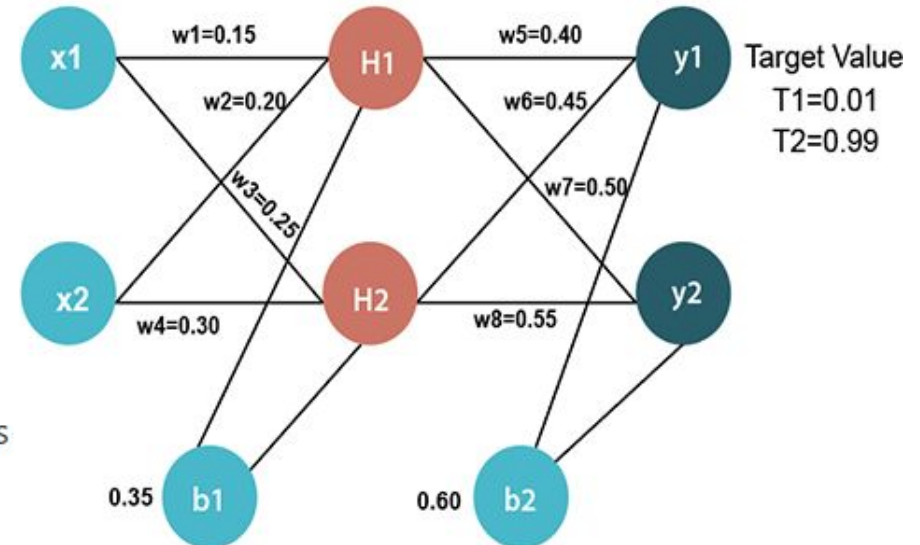
Example

Now, we will calculate the updated weight $w5_{new}$ with the help of the following formula

$$\begin{aligned}w5_{new} &= w5 - \eta \times \frac{\partial E_{total}}{\partial w5} \text{ Here, } \eta = \text{learning rate} = 0.5 \\&= 0.4 - 0.5 \times 0.0821670407 \\ \mathbf{w5_{new} = 0.35891648} \dots \dots \dots (12)\end{aligned}$$

In the same way, we calculate $w6_{new}$, $w7_{new}$, and $w8_{new}$ and this will give us the following values

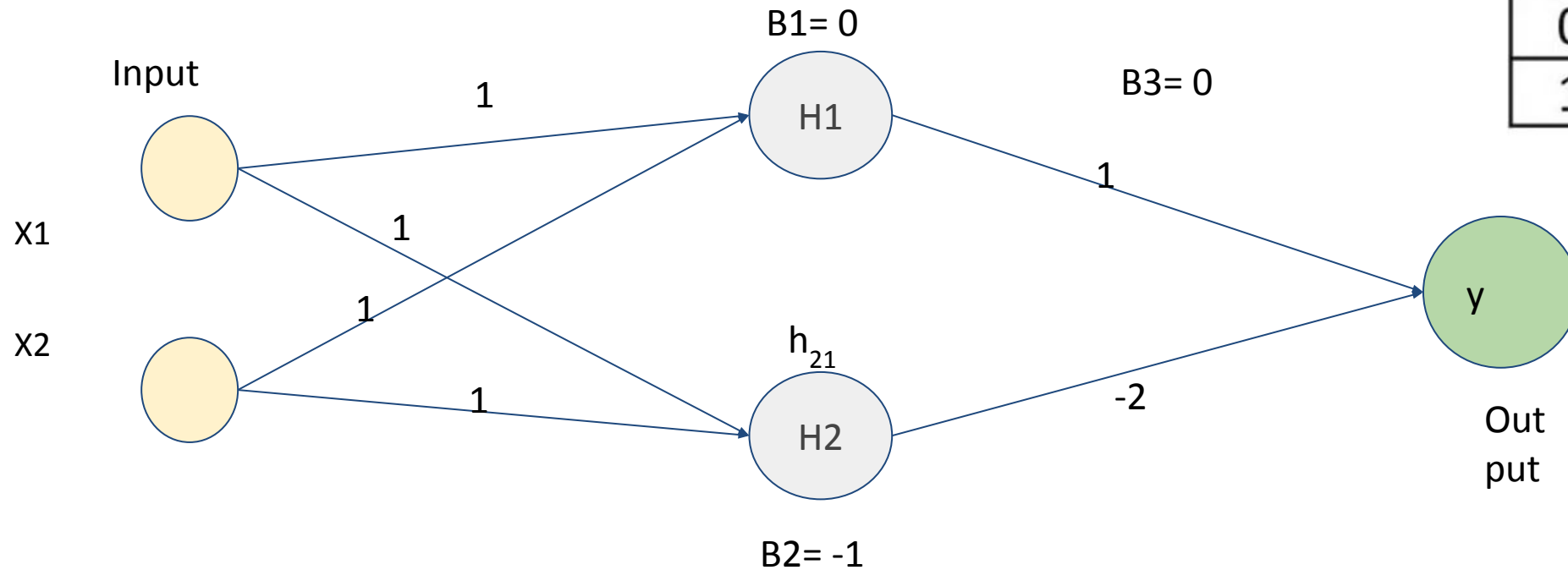
$$\begin{aligned}\mathbf{w5_{new} = 0.35891648} \\ \mathbf{w6_{new} = 0.408666186} \\ \mathbf{w7_{new} = 0.511301270} \\ \mathbf{w8_{new} = 0.561370121}\end{aligned}$$



Backpropagation

- Steps
 - Initialize weights to small random numbers, associated with biases
 - Propagate the inputs forward (by applying activation function)
 - Backpropagate the error (by updating weights and biases)
 - Terminating condition (when error is very small, etc.)

XOR using MLP



A	B	Output
0	0	0
1	0	1
0	1	1
1	1	0

References

- Chapter 5, Deep learning book by MIT
- Chapter 4, Neural Networks and Learning Machines, Haykin
- Chapter 5, Pattern Recognition and Machine Learning, Bishop

Code

- <https://colab.research.google.com/github/rabbanyk/comp551-notebooks/blob/master/MLP.ipynb>

Thank You 😊